

TEACHER REFERENCE GUIDE

Micro:bit Radio Heist – Hacker Edition

Masterclass reference & key difficulty points

** We recommend building the code yourself by following the [Solution slide](#), even if you don't fully understand it yet. The act of building it will help you understand how it works.*

Workshop Overview

This workshop guides students through 5 challenges where they progressively learn to work with the micro:bit and radio signals. The storyline is set in "The Matrix" and students are "Agents" who must find a hidden treasure.

Phase	Challenge	Students learn
1	Hacker-ID	Input (buttons), output (LED icons), timing (pause/clear)
Pop-up	System Test	Set up radio and test receiving (teacher sends test signal)
2	Receiving radio signals	Radio group setup, on received number, variables
Pop-up	Team Formation	Hand out group numbers via envelopes
3	Team Search	Combine sending + receiving, walk around to find team partner
4	Hacking The Matrix	For-loops, variables, systematic testing (brute force)
5	Find the treasure	Signal strength (RSSI), LED patterns, sound

Challenge 1: Hacker-ID

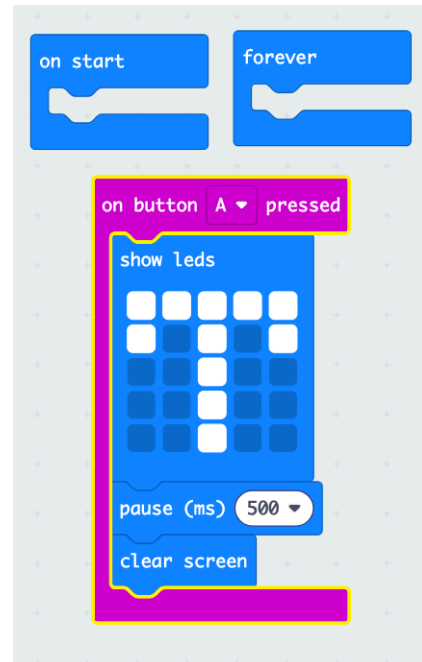
Goal: Students learn to use buttons as input and LED icons as output.

What students need to do

- When pressing a button (A or B), an ID icon appears on the screen
- After displaying, the screen must clear again (clearScreen)
- Optionally add a pause so the icon stays visible briefly

MakeCode blocks used

- on button pressed (A, B, or A+B)
- show icon – choose an icon as their Agent-ID
- pause – wait before clearing the screen
- clearScreen – clears the LED display



Teacher notes

This is the first introduction to MakeCode. Walk around and help with the interface.

Show students where to find blocks in the toolbox (Input, Basic categories).

The icon they choose here becomes their 'Agent-ID' – it comes back in later challenges!

Let students experiment freely before showing the solution.

Difficult parts / where students get stuck

Students forget clearScreen → the icon stays on after the button press

Block order matters: show icon → pause → clearScreen (not the other way around!)

Difference between button A, button B, and A+B can be confusing

Some students don't know how to transfer code to the micro:bit (download process)

Optional extra task

- Create an animation of at least 3 icons that only starts when pressing A+B
- This is their personal hacker signature

Challenge 2: Receiving radio signals

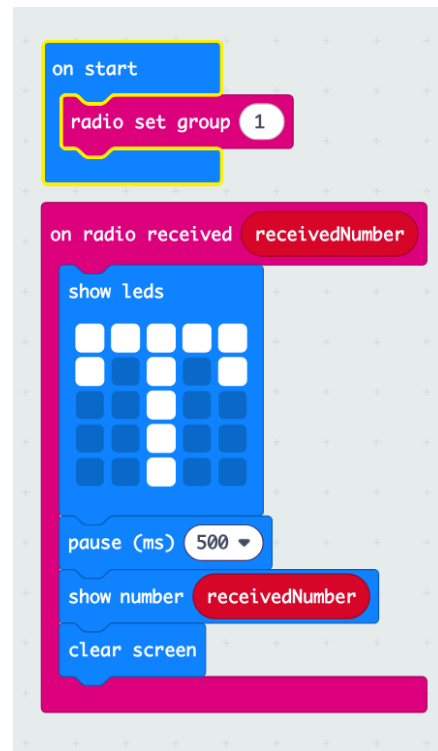
Goal: Students learn to set up the radio and receive + display a signal.

What students need to do

- Set the radio group to group 1 (set group 1)
- Use 'on received number' to catch the signal
- Display the Agent-ID icon first, then the received number (showNumber)

MakeCode blocks used

- set group – MUST be set to 1 (the group The Matrix broadcasts on)
- on received number – this block runs automatically when a signal comes in
- show icon – display Agent-ID
- show number – display the received number (variable 'receivedNumber')



Teacher notes

IMPORTANT: The teacher/facilitator must run The Matrix micro:bit that sends a number on group 1.

Verify all students have set their group to 42!

Explain that 'receivedNumber' is a variable – a container that holds the number.

Show how to drag the variable from the block into other blocks.

Difficult parts / where students get stuck

Forgot to set group or wrong group number → nothing comes in

Variable concept is new: students don't understand that 'receivedNumber' contains the number

Students confuse 'on start' with 'on received number' – explain the difference (event vs. one-time)

set group must be inside 'on start', NOT inside 'on received number'

Optional extra task

- Silent Listener: only show the icon if the number is > 10 (use an if-block)
- Introduces conditional logic – only for faster students

Interlude: Team Formation

Teacher preparation

Prepare envelopes with group numbers beforehand (e.g. 2 through 10).
Stick these under the laptops BEFORE the class starts.
Each group number appears exactly 2 times (pairs).
Group 1 is reserved for The Matrix – do NOT use this number for students!
Tell students to keep their number secret.

Challenge 3: Team Search – sending and receiving signals

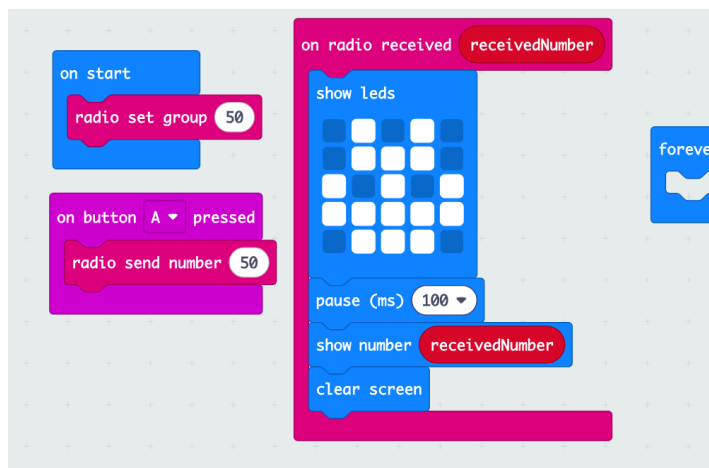
Goal: Students learn to both send and receive, and find their team partner.

What students need to do

- Change the radio group to their own group number (no longer 42!)
- On button press: send the group number with sendNumber
- On receiving: show Agent-ID + received number
- Walk around the classroom and test with others until you find your partner

MakeCode blocks used

- set group – now set to their OWN group number (from the envelope)
- on button pressed – trigger to send
- send number – sends the group number
- on received number – receives signals from others
- show icon + show number – display result
- clearScreen + pause – timing



Teacher notes

This is the first time sending AND receiving are in the same program.
Let students physically walk around – this makes it interactive and fun!
Verify students have changed their group from 1 to their own number.
When two students receive the same number as they send, they're a team.

Difficult parts / where students get stuck

Group still set to 1 instead of their own number → everyone receives everything

Confusion between sendNumber and the number to send (what value to enter?)

Overwriting Challenge 2 code: students need to keep the receiving code!

Students who finish early may disrupt others – have them turn off radio or do the optional task

Optional extra task

- Handshake Protocol: send 'ACK' back after receiving, show a checkmark when 'ACK' is received
- Uses send string and on received string – good for advanced students

Challenge 4: Hacking The Matrix

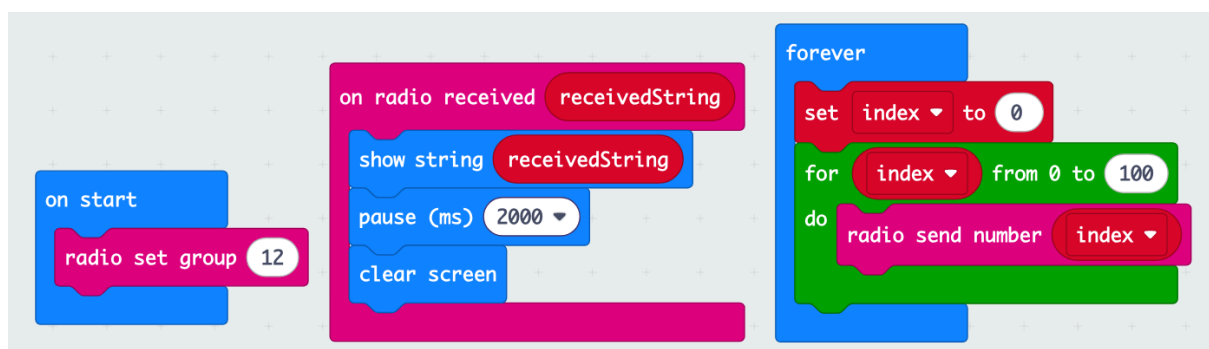
Goal: Students use a for-loop to systematically find the secret key.

What students need to do

- The Matrix listens on their group number and only responds to the correct number
- Students must send numbers and check if The Matrix responds
- Use a for-loop to try all possible numbers (brute force approach)
- When The Matrix responds with a string: that's the clue for the treasure

MakeCode blocks used

- set group – their group number
- for loop – to systematically try numbers (e.g. 0 through 100)
- sendNumber – send each number
- on received string – catch The Matrix's response
- showString – display the receivedString as response
- Variables (index) – the counter of the for-loop
- pause – wait between sends (otherwise it goes too fast)



Teacher notes

THIS IS THE HARDEST CHALLENGE. Be prepared for many questions.

Explain the concept of 'brute force': you simply try everything until something works.

The index variable MUST start at 0 (mentioned in the on-screen tip).

The Matrix must be listening on their group number and only respond to the key!

Verify The Matrix program is running correctly before this challenge starts.

Difficult parts / where students get stuck

For-loop concept is completely new – students don't know where to start

Index not set to 0 → loop starts wrong or doesn't work

No pause in the loop → micro:bit sends everything at once, misses the response

Confusion between sendNumber (inside the loop) and on received string (separate event block)

Students try to put the loop INSIDE the on received block – that doesn't work

String vs. Number confusion: The Matrix sends back a STRING, not a number

Challenge 5: Find the treasure

Goal: Students build a hot/cold detector based on signal strength.

What students need to do

- Use signal strength (RSSI) to determine how close the treasure is
- The stronger the signal, the more LEDs light up or the higher the beep tone
- Walk around with the micro:bit to physically locate the treasure

MakeCode blocks used

- Signal strength / RSSI – value indicating how strong the radio signal is
- LED patterns – visual feedback (more LEDs = closer)
- Sound / beep tone – auditory feedback (optional)
- The program is provided as a blueprint

Teacher notes

This is more of a physical activity – the program is largely provided.

The 'treasure' must be hidden beforehand with a transmitting micro:bit.

RSSI values are negative: -40 is closer than -80.

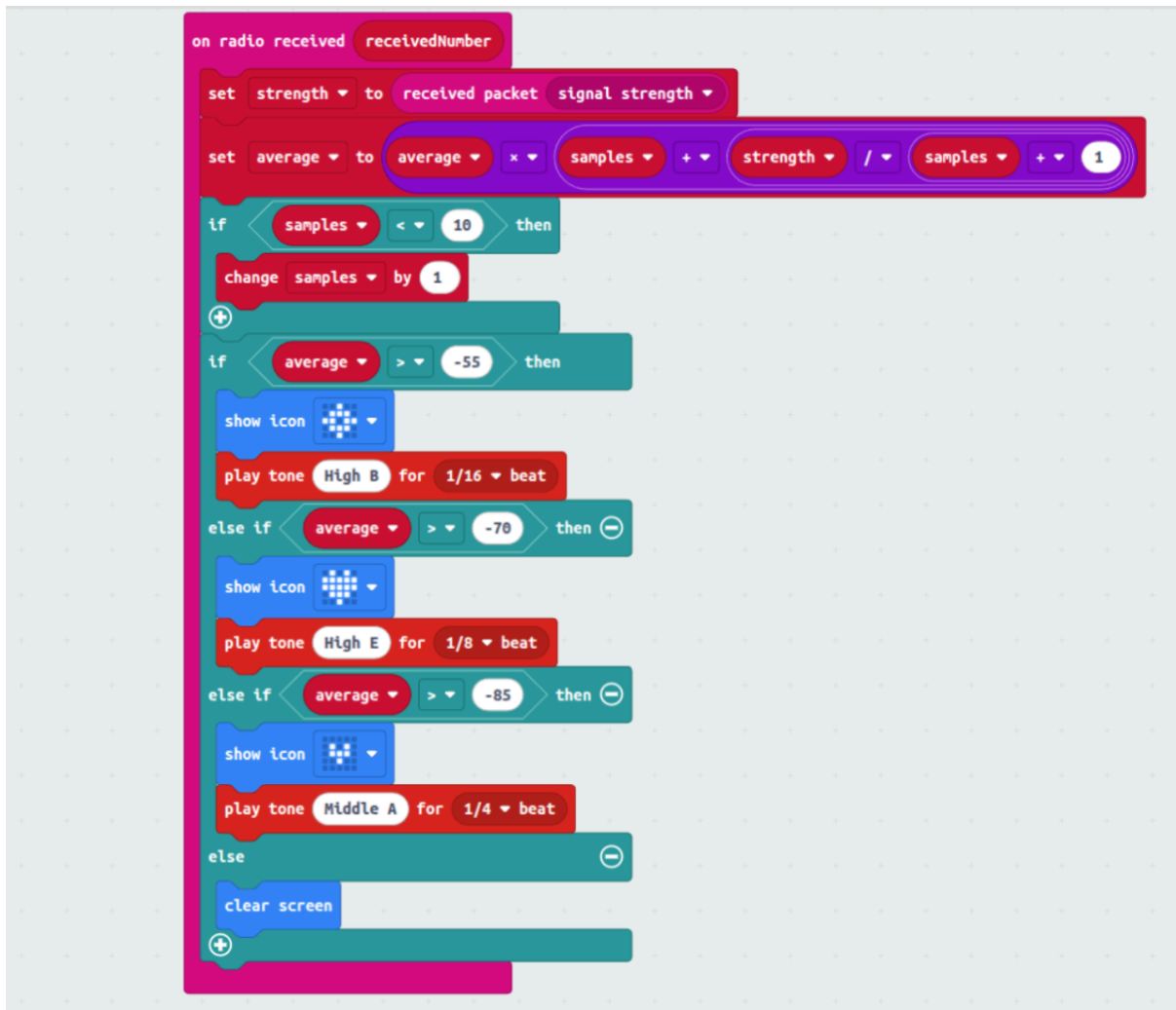
Make sure there's enough space for students to walk around.

Difficult parts / where students get stuck

RSSI concept is abstract – explain with analogy: WiFi bars on your phone

Negative numbers: higher = stronger (-40 > -80, so -40 means closer)

Code is provided, but students still need to flash it onto their micro:bit



Preparation Checklist

Have ready beforehand

- Envelopes with group numbers (2x each number, e.g. 2–15) stuck under laptops
- The Matrix micro:bit programmed and tested (broadcasts on group 1 + listens per group number)
- Treasure micro:bit programmed and hidden for Challenge 5
- All student micro:bits tested and working
- MakeCode open on all laptops (makecode.microbit.org)
- Blueprint/code for Challenge 5 printed or available digitally

Common problems and solutions

Problem	Solution
Micro:bit receives nothing	Check the radio group. Is it set to the correct number? Is set group inside 'on start'?
Code doesn't work after downloading	Check if the micro:bit is properly connected. Try re-flashing.
For-loop does nothing (Ch.4)	Check if index is set to 0. Is there a pause in the loop? Is sendNumber correct?
Everyone receives everything	Students are still on group 1. Have them set their own group number.
Matrix doesn't respond (Ch.4)	Check if the Matrix micro:bit is on and listening on the correct group number.

Glossary for Teachers

Term	Explanation
Radio group	A channel on which micro:bits communicate. Only micro:bits on the same group can hear each other.
Variable	A named container that holds a value (e.g. receivedNumber).
Event	A block that runs automatically when something happens (e.g. button press, signal received).
For-loop	Repeats code a certain number of times. The index counts up with each iteration.
Brute force	Trying all possibilities until you find the right one.
RSSI	Received Signal Strength Indicator – shows how strong a radio signal is. Higher (less negative) value = closer.
sendNumber	Sends a number via radio to all micro:bits on the same group.
sendString	Sends text via radio.
ACK	Acknowledged – confirmation that a message was received (just like in real networks).